

Newcastle University

School of Computing

CSC8110 – Cloud Computing

Report 2025/26

Submitted by;

Karthik S

Introduction:

This report describes how a cloud-native application stack is implemented. The main goals were to install a Java-based web application, design a comprehensive monitoring pipeline (Prometheus & Grafana), and create a custom containerized load generator to test system performance. The solution demonstrates the integration of custom Docker images, Kubernetes orchestration, and observability tools.

Implementation Details:

Task-1: Application Deployment & Dashboard

Objective: Deploy the Kubernetes Dashboard and the javabenchmarkapp with specific access requirements.

- **Kubernetes Dashboard:** As stated in the requirements, I've avoided the standard MicroK8s dashboard add-on, instead deploying the dashboard manually. To secure access, I created a ServiceAccount named admin-user and a ClusterRoleBinding to the cluster-admin role. This allowed for the generation of a secure bearer token for login.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: admin-user
  namespace: kubernetes-dashboard
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: admin-user
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: admin-user
  namespace: kubernetes-dashboard
```

Dashboard-admin.yaml

http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:/proxy/#/login

90%

Kubernetes Dashboard

☒ Token

Every Service Account has a Secret with valid Bearer Token that can be used to log in to Dashboard. To find out more about how to configure and use Bearer Tokens, please refer to the [Authentication](#) section.

☐ Kubeconfig

Please select the kubeconfig file that you have created to configure access to the cluster. To find out more about how to configure and use kubeconfig file, please refer to the [Configure Access to Multiple Clusters](#) section.

Enter token *

Sign In

← → ↺

http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:/proxy/#/workloads?namespace=default

⌵ ⌵ ⌵

kubernetes

default

Search

+ 🔔 ⌵

Workloads

Workloads

Cron Jobs

Daemon Sets

Deployments

Jobs

Pods

Replica Sets

Replication Controllers

Stateful Sets

Service

Ingresses

Ingress Classes

Services

Config and Storage

Config Maps

Persistent Volume Claims

Secrets

Storage Classes

Workload Status

Running: 2

Deployments

Running: 2

Pods

Running: 3

Replica Sets

Deployments

Name	Images	Labels	Pods	Created ↑	
load-generator	localhost:32000/load-generator:latest	-	1 / 1	34 minutes ago	⋮
java-benchmark-deployment	ncicloudcomputing/javabenchmarkapp	app: java-benchmark	1 / 1	2 hours ago	⋮

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑	
java-benchmark-deployment-7fd65b5599-4xn96	ncicloudcomputing/javabenchmarkapp	app: java-benchmark pod-template-hash: 7fd65b5599	csc8110-kdocker	Running	0	-	-	11 minutes ago	⋮
load-generator-6bc9698f8f-vptpb	localhost:32000/load-generator:latest	app: load-generator pod-template-hash: 6bc9698f8f	csc8110-kdocker	Running	0	-	-	35 minutes ago	⋮

Replica Sets

Name	Images	Labels	Pods	Created ↑	
java-benchmark-deployment-7fd65b5599	ncicloudcomputing/javabenchmarkapp	app: java-benchmark pod-template-hash: 7fd65b5599	1 / 1	11 minutes ago	⋮
load-generator-6bc9698f8f	localhost:32000/load-generator:latest	app: load-generator pod-template-hash: 6bc9698f8f	1 / 1	35 minutes ago	⋮
java-benchmark-deployment-6b86c498f5	ncicloudcomputing/javabenchmarkapp	app: java-benchmark pod-template-hash: 6b86c498f5	0 / 0	2 hours ago	⋮

Kubernetes Dashboard

- **Web Application:** I deployed the nclcloudcomputing/javabenchmarkapp image. To satisfy the requirement of host accessibility at <http://localhost:30000/primecheck>, I configured a NodePort Service.
 - Configuration: The service maps the container's internal port 8080 to the host's port 30000.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-benchmark-deployment
  labels:
    app: java-benchmark
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-benchmark
  template:
    metadata:
      labels:
        app: java-benchmark
    spec:
      containers:
        - name: java-benchmark
          image: nclcloudcomputing/javabenchmarkapp
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: java-benchmark-service
spec:
  type: NodePort
  selector:
    app: java-benchmark
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 30000
```

Java-app.yaml

```
student@CSC8110-KDocker:~$ curl -v http://localhost:30000/primecheck
* Trying 127.0.0.1:30000...
* Connected to localhost (127.0.0.1) port 30000 (#0)
> GET /primecheck HTTP/1.1
> Host: localhost:30000
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200
< Content-Type: text/plain; charset=UTF-8
< Content-Length: 12
< Date: Tue, 09 Dec 2025 15:40:36 GMT
<
* Connection #0 to host localhost left intact
```

Proof of Access (Curl)

```
student@CSC8110-KDocker:~$ curl http://localhost:30000/primecheck
Time: 238 msstudent@CSC8110-KDocker:~$
```

Primecheck Output (via Curl)

Task-2: The Monitoring Stack

Objective: Deploy Prometheus, Node Exporter, Kube-state-metrics and Grafana to monitor cluster health.

```
student@CSC8110-KDocker:~$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
kube-state-metrics-5698cb54d9-lkpw6 1/1     Running   5 (36m ago) 5d1h
cadvisor-g99wn                       1/1     Running   5 (36m ago) 5d
grafana-659df76d94-bhtfs            1/1     Running   5 (36m ago) 5d1h
prometheus-deployment-6f689c9749-jgrxd 1/1     Running   5 (36m ago) 4d23h
node-exporter-flbtm                 1/1     Running   6 (36m ago) 5d18h
```

```
student@CSC8110-KDocker:~$ kubectl get deployment -n monitoring
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
kube-state-metrics                 1/1     1             1           5d18h
grafana                             1/1     1             1           5d18h
prometheus-deployment              1/1     1             1           5d18h
```

I consolidated the core monitoring infrastructure into **monitoring-components.yaml**, ensuring all components interact correctly.

- **Prometheus (The Aggregator):** Prometheus was configured via a ConfigMap to scrape targets every 15 seconds. I defined scrape jobs for node-exporter (host metrics), cadvisor (container metrics), and kube-state-metrics.

```

# Prometheus Configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-server-conf
  namespace: monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
    scrape_configs:
      - job_name: 'node-exporter'
        static_configs:
          - targets: ['node-exporter:9100']

      - job_name: 'kube-state-metrics'
        static_configs:
          - targets: ['kube-state-metrics:8080']

      - job_name: 'cadvisor-service'
        metrics_path: /metrics
        static_configs:
          - targets: ['cadvisor:8080']

```

Prometheus-ConfigMap

```

# Prometheus Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus-deployment
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus-server
  template:
    metadata:
      labels:
        app: prometheus-server
    spec:
      serviceAccountName: prometheus
      containers:
        - name: prometheus
          image: prom/prometheus
          args:
            - "--config.file=/etc/prometheus/prometheus.yml"
          ports:
            - containerPort: 9090
          volumeMounts:
            - name: prometheus-config-volume
              mountPath: /etc/prometheus/
      volumes:
        - name: prometheus-config-volume
          configMap:
            defaultMode: 420
            name: prometheus-server-conf

```

Prometheus-Deployment

```
# Prometheus Service
apiVersion: v1
kind: Service
metadata:
  name: prometheus-service
  namespace: monitoring
spec:
  selector:
    app: prometheus-server
  ports:
    - port: 9090
      targetPort: 9090
```

Prometheus-Service

- **RBAC Configuration:** To allow Prometheus to discover pods and nodes dynamically, I implemented a ClusterRole and ClusterRoleBinding granting it get, list and watch permissions on core resources.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
- apiGroups: [""]
  resources: ["nodes", "nodes/proxy", "services", "endpoints", "pods"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get"]
- nonResourceURLs: ["/metrics"]
  verbs: ["get"]
```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: prometheus
  namespace: monitoring

```

Monitoring-rbac.yaml

- **Node Exporter (Host Metrics):** To ensure scalability, I deployed Node Exporter as a DaemonSet. This ensures that if the cluster scales to multiple nodes, a monitoring agent is automatically provisioned on every new host without manual intervention.

```

# Node Exporter (DaemonSet)
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter
          ports:
            - containerPort: 9100

```

Node Exporter – DaemonSet

- **Kube-State-Metrics (Cluster State):** I deployed kube-state-metrics to monitor the state of Kubernetes objects (e.g., Pod status, Deployment readiness) rather

than their resource usage. This provides critical context for the dashboard, distinguishing between a “crashing” pod and a “busy” pod.

```
# Kube State Metric Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kube-state-metrics
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kube-state-metrics
  template:
    metadata:
      labels:
        app: kube-state-metrics
    spec:
      serviceAccountName: prometheus
      containers:
        - name: kube-state-metrics
          image: registry.k8s.io/kube-state-metrics/kube-state-metrics:v2.0.0
          ports:
            - containerPort: 8080
```

Kube-state-metrics Deployment

```
student@CSC8110-KDocker:~$ kubectl get deployment kube-state-metrics -n monitoring
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
kube-state-metrics      1/1     1            1           5d18h
```

Deployment stats – Kube-state-metrics

- **cAdvisor (Container Metrics):** To fulfill the Task 4 requirements of monitoring specific container resource usage, I deployed cAdvisor.
 - **MicroK8s Adaptation:** As MicroK8s uses containerd rather than the standard Docker daemon, the default cAdvisor configuration often fails. I explicitly mounted the socket at /var/run/microk8s-socket/containerd.sock to ensure successful metric collection.

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: cadvisor
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: cadvisor
  template:
    metadata:
      labels:
        app: cadvisor
    spec:
      containers:
        - name: cadvisor
          image: gcr.io/cadvisor/cadvisor:v0.47.2
          securityContext:
            privileged: true
          ports:
            - containerPort: 8080
          args:
            - --containerd=/var/run/microk8s-socket/containerd.sock
            - --docker_only=false
            - --disable_metrics=disk
          volumeMounts:
            - name: rootfs
              mountPath: /rootfs
              readOnly: true
            - name: microk8s-socket-dir
              mountPath: /var/run/microk8s-socket
              readOnly: true
            - name: sys
              mountPath: /sys
              readOnly: true

```

```

      volumes:
        - name: rootfs
          hostPath:
            path: /
        - name: microk8s-socket-dir
          hostPath:
            path: /var/snap/microk8s/common/run
        - name: sys
          hostPath:
            path: /sys
---
apiVersion: v1
kind: Service
metadata:
  name: cadvisor
  namespace: monitoring
spec:
  selector:
    app: cadvisor
  ports:
    - port: 8080
      targetPort: 8080

```

Cadvisor YAML

- **Grafana (Visualization):** Grafana was exposed via NodePort 31000 and configured with Prometheus as it's data source to visualize these streams.
Grafana URL used – <http://localhost:31000/>

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      labels:
        app: grafana
    spec:
      containers:
        - name: grafana
          image: grafana/grafana
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: grafana-service
  namespace: monitoring
spec:
  type: NodePort
  selector:
    app: grafana
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 31000
```

Grafana.yaml

 **prometheus**

Type: Prometheus

 **Settings**  Dashboards  Permissions  Insights  Cache

Name ⓘ Default ☒

Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, [view the documentation](#).

Fields marked with * are required

Connection

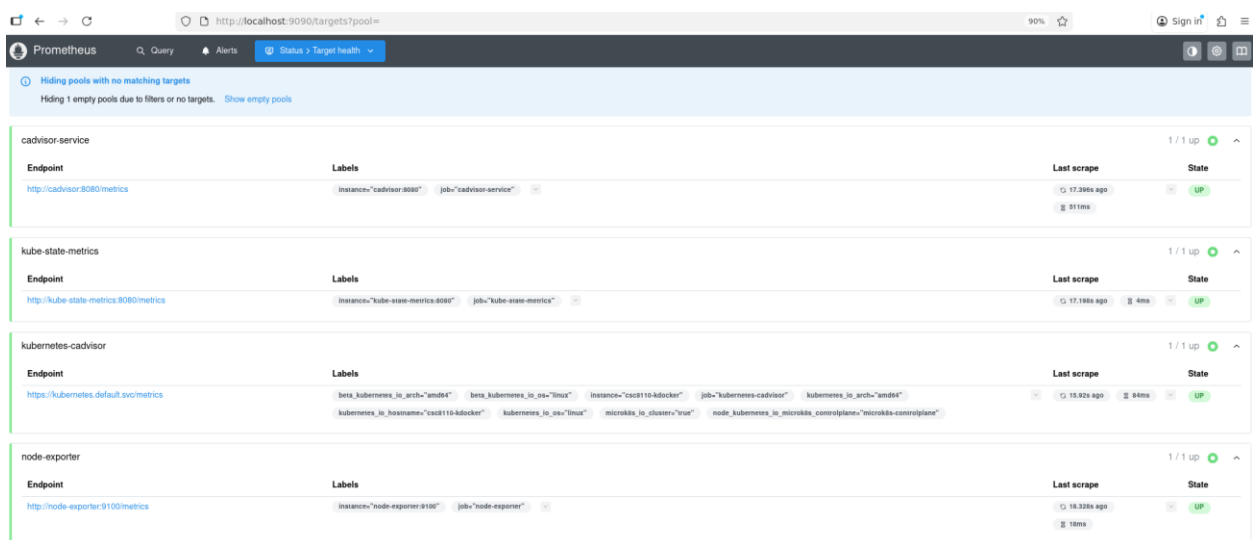
Prometheus server URL * ⓘ



Grafana UI and Adding Data Source (Prometheus)

- The following image shows the Prometheus Targets page (which shows what Prometheus is scraping and if it's healthy) and how to access it.

```
student@CSC8110-KDocker:~$ kubectl port-forward svc/prometheus-service 9090:9090 -n monitoring
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
```



Prometheus/Targets – Access Command & Results

Task-3: Load Generator

Objective: Develop and deploy a custom load generator.

- **Application Logic:** I developed a Python script (`load_generator.py`) to stress-test the application.
 - **Configurability:** It uses `os.getenv` to ingest TARGET and FREQUENCY variables, making the image reusable.
 - **Resilience:** The script includes a try/except block to catch `requests.exceptions.RequestException`. Any request exceeding 10 seconds is explicitly counted as a failure.

```

# Configuration via Environment Variables
target = os.getenv('TARGET', 'http://localhost:30000/primecheck')
try:
    frequency = float(os.getenv('FREQUENCY', '5'))
except ValueError:
    print("Invalid FREQUENCY setting. Defaulting to 5.")
    frequency = 5.0

print(f"Load Generator Starting")
print(f"Target: {target}")
print(f"Frequency: {frequency} req/s")

# Metrics
total_requests = 0
failures = 0
total_response_time = 0

interval = 1.0 / frequency

while True:
    start_time = time.time()
    total_requests += 1

    try:
        # Generate Request with 10s Timeout
        response = requests.get(target, timeout=10)

        # Check status code
        if response.status_code == 200:
            total_response_time += response.elapsed.total_seconds()
        else:
            failures += 1

    except requests.exceptions.RequestException:
        # Timeout or connection error counts as failure
        failures += 1

# Calculate & Print Metrics
avg_rt = 0
successes = total_requests - failures
if successes > 0:
    avg_rt = total_response_time / successes

print(f"Total: {total_requests} | Failures: {failures} | Avg Response Time: {avg_rt:.4f}s")

# Frequency Control
elapsed = time.time() - start_time
sleep_time = interval - elapsed
if sleep_time > 0:
    time.sleep(sleep_time)

```

Load_generator.py

- Dockerization & Registry:** The Application was packaged using a python:3.9-slim base image. I utilized a local insecure registry at localhost:32000.
 - Image built as load-generator.

```
FROM python:3.9-slim

# Install dependencies
RUN pip install requests

# Add the script
COPY load_generator.py /app/load_generator.py
WORKDIR /app

# Set unbuffered output so logs show up immediately in Kubernetes
ENV PYTHONUNBUFFERED=1

# Command to run
CMD ["python", "load_generator.py"]
```

Dockerfile

2. Tagged for local registry: localhost:32000/load-generator.

```
student@CSC8110-KDocker:~$ docker build -t localhost:32000/load-generator:latest .
[+] Building 11.8s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 331B                             0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.0s
=> [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261 6.5s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261 0.0s
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa07504 1.29MB / 1.29MB 0.6s
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83 13.88MB / 13.88MB 2.7s
=> => sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acd 10.36kB / 10.36kB 0.0s
=> => sha256:dad5b29e3506c35e0fd222736f4d4ef25d21b219acdd73f7bb41 1.74kB / 1.74kB 0.0s
=> => sha256:085da638e1b8a449514c3fda83ff50a3bffa4418b050cfacd87 5.40kB / 5.40kB 0.0s
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab 29.78MB / 29.78MB 2.2s
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c4500783 251B / 251B 1.0s
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191 2.3s
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3 0.1s
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710aff 1.5s
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c45007831 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 1.44kB                                0.0s
=> [2/4] RUN pip install requests                             3.0s
=> [3/4] COPY load_generator.py /app/load_generator.py         0.0s
=> [4/4] WORKDIR /app                                          0.0s
=> => exporting to image                                          0.1s
=> => exporting layers                                           0.1s
=> => writing image sha256:0ce3ae1c34e70cf85d162687e43febb5bbd265ff5e10e805da9876 0.0s
=> => naming to localhost:32000/load-generator:latest          0.0s
```

Docker Build

3. Pushed successfully.

```
student@CSC8110-KDocker:~$ docker push localhost:32000/load-generator:latest
The push refers to repository [localhost:32000/load-generator]
5f70bf18a086: Pushed
3aeffc0c0b7: Pushed
a94272f893e3: Pushed
c8f6b54339a8: Pushed
298992e09a03: Pushed
4f237755fbae: Pushed
d7c97cb6f1fe: Pushed
latest: digest: sha256:4ef8ce31a980c80d17d20285ce9d5ae7707955e2ecbb6e7e60255a2f99c43487 size: 1783
```

Docker Push

```
student@CSC8110-KDocker:~$ curl http://localhost:32000/v2/_catalog
{"repositories":["load-generator"]}
```

Registry Proof – Image exists in the local registry

- **Deployment:** The deployment manifest injects the target URL and a frequency of 20 req/s via Environment Variables.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: load-generator
spec:
  replicas: 1
  selector:
    matchLabels:
      app: load-generator
  template:
    metadata:
      labels:
        app: load-generator
    spec:
      containers:
        - name: load-generator
          image: localhost:32000/load-generator:latest
          imagePullPolicy: Always
          env:
            - name: TARGET
              value: "http://java-benchmark-service.default.svc.cluster.local:8080/primecheck"
            - name: FREQUENCY
              value: "20"
```

Load-generator-deployment.yaml

```
student@CSC8110-KDocker:~$ kubectl get deployment load-generator
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
load-generator      1/1     1            1           5d17h

student@CSC8110-KDocker:~$ kubectl apply -f load-generator-deployment.yaml
deployment.apps/load-generator created
student@CSC8110-KDocker:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
java-benchmark-deployment-6b86c498f5-zhcmb   1/1     Running   0          123m
load-generator-6bc9698f8f-vptpb             1/1     Running   0          10s

student@CSC8110-KDocker:~$ kubectl logs -l app=load-generator
Total: 171 | Failures: 0 | Avg Response Time: 0.1431s
Total: 172 | Failures: 0 | Avg Response Time: 0.1431s
Total: 173 | Failures: 0 | Avg Response Time: 0.1431s
Total: 174 | Failures: 0 | Avg Response Time: 0.1431s
Total: 175 | Failures: 0 | Avg Response Time: 0.1431s
Total: 176 | Failures: 0 | Avg Response Time: 0.1431s
Total: 177 | Failures: 0 | Avg Response Time: 0.1431s
Total: 178 | Failures: 0 | Avg Response Time: 0.1430s
Total: 179 | Failures: 0 | Avg Response Time: 0.1431s
Total: 180 | Failures: 0 | Avg Response Time: 0.1431s
```

Deployment and Logs from Load-Generator

Task-4: Benchmarking Results

Obejctive: Visualize the impact of the load generator on the javabenchmarkapp.

The following dashboards visualize the metrics captured during the load test.



- **Memory Usage:** The graph shows the working set bytes increasing as the JVM allocates heap memory for the calculations.
- **CPU Usage:** A distinct spike is visible correlating exactly with the load generator's active window. The usage rises sharply from idle to approximately 0.96 cores.

Discussion of Results:

Performance Analysis:

The benchmarking results provide critical information about the application architecture.

- **Single-Threaded Bottleneck:** Although there was no CPU restrictions imposed on the number of cores of the user container *javabenchmarkapp* to restrict access to all host cores, the CPU usage always stabilized at 0.96 cores. This

confirms that the primecheck is single-threaded meaning that it is physically unable to utilize more than one CPU core.

- **Kernel Overhead:** The application utilized almost 96% of a single core's capacity. The remaining 4% is taken up by the kernel overhead, specifically context switching and network stack processing required to process the incoming HTTP requests at 20 req/s.
- **Scaling Implication:** Due to the single-threaded nature of the app, the "Vertical Scaling" (increasing the size/speed of the CPU core) will not improve performance. "Horizontal Scaling" (increasing *replicas* count in the Deployment) is the required strategy to handle higher loads, as it allows the workload to be distributed across multiple cores by running parallel instances of the application.

Critical Review of Monitoring Architecture:

The monitoring stack implementation needed certain adjustments to the MicroK8s environment, therefore, highlighting the distinction between the traditional Kubernetes distribution based on Docker and the one based on containerd

- cAdvisor Adaptation [**Technical Challenges & Solutions**]: A standard cAdvisor deployment attempts to mount `/var/run/docker.sock` to query container statistics. Since MicroK8s has containerd as its container runtime, a standard deployment would fail to return metrics. As a result, the DaemonSet was changed expressly to support the socket being mounted at `/var/run/microk8s-socket/containerd.sock`, which was an important change that was necessary to satisfy the requirement of container-level resource tracing.
- DaemonSet Strategy: Node-exporter was not deployed as a Deployment or Sidecar, it was deployed as a DaemonSet indicating the architectural best practice of monitoring at the node-level. This method will ensure that only one copy of the exporter executes on each node in the cluster automatically. In the case that the cluster grows to include say 10 nodes, and the DaemonSet controller will automatically offer monitoring on the

new nodes without any human interaction and thus provide smooth, gapless observability.

Development Lifecycle Decisions:

- **Insecure Registry Trade-off:** As per the conditions of the task, local registry at localhost:32000 was utilized. Although a production environment would provide a security risk without TLS encryption and authentication, the local loopback address has been used in this assessment, which removed the overhead of managing the use of SSL certificates and thus provided speed in iteration on the load generator image.
- **Load Generator Logic:** The custom load generator is designed to be a synchronous, blocking request model with a safety mechanism which is a timeout. As seen in the code, the *try...except* block ensures that if the server becomes unresponsive (response time >10s), the generator does not hang indefinitely. Rather, it crashes quickly and logs the failure, providing an accurate availability metric alongside performance latency.

Conclusion:

This assessment successfully demonstrated the end-to-end implementation of a cloud-native application stack. By integrating a Java web application with a custom-built observability pipeline and a load generation tool, the project met all functional requirements outlined in the assessment specification.

Key Achievements:

1. **Infrastructure as Code:** The entire stack, including the application, dashboard, and monitoring tools, was defined using declarative YAML scripts, ensuring reproducibility.
2. **Platform Adaptability:** The solution successfully addressed specific MicroK8s constraints, particularly regarding the secure access of the Dashboard via

RBAC tokens and the reconfiguration of cAdvisor for the *containerd* runtime.

3. Data-Driven Insight: The integration of Prometheus and Grafana transformed raw system data into actionable insights. The visualization of metrics allowed for the definitive identification of the application's single-threaded architectural limitation.

In conclusion, the system not only hosts the application but provides the necessary means to monitor its health, benchmark its performance, and make informed decisions regarding its future scaling requirements.